Computer Algebra System

Technical Guide

Dennis Hone

Contents

Syntax 3

Context Free Grammar 3

Built-In Functions and Constants 4

Constants 4

Trigonometric Functions 4

Hyperbolic Functions 5

Common Mathematical Functions 5

Calculus and Special Functions 5

Deriv Function 6

nDeriv Function 6

Integ Function 6

Diff Function 6

LambertW Function 7

Sum Function 7

Product Function 7

Algebraic Functions 8

Simplify Function 8

Expand Function 9

User-Defined Symbols 9

User-Defined Functions 9

User-Defined Variables 9

Syntax

For functions and for mathematical expressions, this CAS uses a syntax common to most mathematical expression parsers. However, implied multiplication is not supported.

The supported operators are listed below. A higher number indicates higher precedence, so 4 takes precedence over 3.

Operator	Name	Precedence
^	Power	6
-	Negation	5
*	Multiplication	4
/	Division	4
+	Addition	3
-	Subtraction	3
=	Equality	2
:=	Assignment	1

Context Free Grammar

The context-free grammar used for parsing is as follows:

line → expr | assign
assign → IDENT ASSIGN expr
expr → LPAR expr RPAR
expr → NEG expr
expr → expr op expr
expr → func
expr → atom
atom → IDENT | NUM
func → IDENT LPAR params RPAR
func → IDENT LPAR RPAR
params → expr
params → expr COMMA params
op → PLUS | MINUS | DIV | MULT | POW | EQ

An identifier, listed as IDENT above, must be a letter, followed by any number of letters, digits, or underscores. Identifiers are case sensitive. For example: b, variable2, my_function.

A numeral, listed as NUM above, is composed of 3 components: the integer component, the fractional component, and the exponential component. Each component may contain any number of digits. There must be a period between the integer and fractional

component whenever both are present. The 0 may be omitted from the integer or fractional component, as long as the other component is present. For example: ".2" instead of "0.2" or "1." instead of "1.0". To include an exponential component, there must be an E or an e following the integer and/or fractional component. The E can optionally be followed by a "+" or a "-". For example, "1.3E-10", "5.6E38", "9.99E+20". More examples of valid numerals are: "256", "3.1415926", "6.022E23". The sign is not included in this specification because the sign can be changed via the negation operator.

The tokens listed in all caps above, except for IDENT and NUM, correspond to the following symbols:

Token	Symbol
ASSIGN	:=
LPAR	(
RPAR	)
NEG	-
COMMA	,
PLUS	+
MINUS	-
DIV	/
MULT	*
POW	^
EQ	=

Built-In Functions and Constants

Constants

The following constants are defined in the application.

Identifier	Name	Value
pi	Pi	3.1415926535
e	Euler's Number	2.7182818285
inf	Infinity	∞
gamma	Euler-Mascheroni Constant	0.5772156649

Trigonometric Functions

The following trigonometric functions are supported by the application. All trigonometric functions take one parameter and operate in radians.

Function	Description	Example
sin	Trigonometric sine	sin(pi/2)
cos	Trigonometric cosine	cos(pi/4)
tan	Trigonometric tangent	tan(pi/6)
asin	Inverse trigonometric sine	asin(1)
acos	Inverse trigonometric cosine	acos(1/2)
atan	Inverse trigonometric tangent	atan(sqrt(3))

Hyperbolic Functions

The following hyperbolic functions are supported by the application:

Function	Description	Example
sinh	Hyperbolic sine	sinh(ln(2))
cosh	Hyperbolic cosine	cosh(1)
tanh	Hyperbolic tangent	tanh(2ln(3))
asinh	Inverse hyperbolic sine	asinh(exp(1))
acosh	Inverse hyperbolic cosine	acosh(3)
atanh	Inverse hyperbolic tangent	atanh(1)

Common Mathematical Functions

Function	Description	Example
pow	Power function	pow(2,2)
exp	Natural exponential function	exp(1)
log	Natural logarithm	log(4)
ln	Natural logarithm	ln(2)
sqrt	Square root	sqrt(2)
fact	Factorial	fact(5)
mod	Modulo	mod(5,2)

Calculus and Special Functions

A quick reference table of supported calculus and special functions is provided below. For further information, please refer to the function's section.

Function	Description	Example
deriv	Derivative	deriv(x^2+x,x)
nderiv	Nth derivative	nderiv(x^n*ln(x),x,2)
integ	Integral	integ(sqrt(1-x^2),x,-1,1)
diff	Numeric derivative	diff(exp(x),x,2)
lambertw	Lambert W	lambertw(1)

sum	Summation	sum(1/k^2,k,1,1E4)
product	Product	product(1+1/k,k,1,10)

Deriv Function

The **deriv** function takes two parameters. The first must be an expression and the second must be a variable name. This function performs first-order symbolic differentiation on the expression.

The following expression performs the below operation: deriv(a^x,x).

$$\frac{d}{dx}(a^x) = a^x \ln a$$

nDeriv Function

The **nderiv** function performs an nth-order derivative on an expression. It takes 3 parameters: an expression, a variable name, and a numeral. The following expression performs the operation shown below: nderiv(x^n,x,3).

$$\frac{d^3}{dx^3}(x^n) = n(n-1)(n-2)x^{n-3}$$

Integ Function

The **integ** function performs numeric integration on an expression between the specified limits. It takes 4 parameters: an expression, a variable name, a numeral, and another numeral. The following expression performs the operation shown below:

integ(exp(-x)*x^5,x,0,1E4)

$$\int_0^{10^4} e^{-x}x^5 dx \approx 120$$

The integral function performs numeric quadrature using the 3-point Newton-Cotes formula. For reference, the formula is included below:

$$\int_{x_1}^{x_3} f(x)dx \approx \frac{h}{3}(f_1 + 4f_2 + f_3)$$

Diff Function

The **diff** function performs numeric differentiation on an expression at a specified point. It takes 3 parameters: an expression, a variable name, and a numeral. The following expression performs the operation shown below: diff(sin(x),x,pi/4).

$$\frac{d}{dx}\sin(x)\Big|^{x=\pi/4} \approx 0.70710678$$

LambertW Function

The **lambertw** function computes the value of the Lambert W function on the principal branch (index 0) at a given point. It takes 1 parameter: an expression. The following expression performs the operation shown below: lambertw(1).

$$W_0(1) \approx 0.56714329$$

The Lambert W function is the inverse function of $f(x) = xe^x$. It satisfies the identities $W_0(x)e^{W_0(x)} = x$ and $W_0(xe^x) = x$. Its derivative is: $W_0'(x) = \frac{W_0(x)}{x\,(1+W_0(x))}$.

Sum Function

The **sum** function performs a summation on an expression between the given limits. It supports both symbolic and numeric results. It performs the following mathematical operation:

$$\sum_{k=a}^{b} f(k)$$

For example, the following expression performs the sum of consecutive squares from 0 to 4: sum(k^2,k,0,4).

$$\sum_{k=0}^{4} k^2 = 30$$

Additionally, the following expression returns the finite power series shown below: sum(x^k,k,0,4).

$$\sum_{k=0}^{4} x^k = 1 + x + x^2 + x^3 + x^4$$

Product Function

The **product** function performs a product on an expression between the given limits. Both symbolic and numeric results are supported. It performs the following mathematical operation:

$$\prod_{k=a}^{b} f(k)$$

For example, the following expression computes the product shown below: product(4*k^2/(4*k^2-1),k,1,1e6).

$$\prod_{k=1}^{10^6} \frac{4k^2}{4k^2-1} \approx 1.570796$$

As another example, the following expression returns a symbolic product: product(x+k,k,0,4).

$$\prod_{k=0}^{4} (x+k) = x(x+1)(x+2)(x+3)(x+4)$$

Algebraic Functions

A quick reference table of the supported algebraic functions is included below. For further information, please refer to the individual function's section.

Function	Description	Example
simplify	Algebraic simplification	simplify(x^2+x+3*x^2+1)
expand	Algebraic expansion	expand((x+1)^6)

Simplify Function

The **simplify** function performs algebraic simplification on an expression. It takes one parameter: an expression. It is capable of recognizing and combining like terms in a polynomial and combining like factors in the monomial components of a polynomial. It converts division by a polynomial into multiplication by a polynomial of negative order.

The following expression will simplify the terms and combine when necessary: simplify(x^2*y/x+2*y+x*y). It performs the following algebraic operation:

$$\frac{x^2y}{x} + 2y + xy = 2xy + 2y$$

The following expression will simplify the first monomial and combine the occurrences of x in the expression: simplify(3*x*y*x+x^2+x^2*y). It performs the following algebraic operation:

$$3xyx + x^2 + x^2y = 4x^2y + x^2$$

Expand Function

The **expand** function performs algebraic expansion on an expression. It takes on parameter: an expression. For example, the following expression performs the expansion below: expand((x-y)*(a+b)).

$$(x-y)(a+b) = xa + xb - ya - yb$$

In the following expression, the function performs the binomial expansion shown below: expand((a+b)^3).

$$(a+b)^3 = a^3 + 3a^2b + 3ab^2 + b^3$$

User-Defined Symbols

User-Defined Functions

To create a user-defined function, the following syntax is used:

[name](a,b,...) := *[expression]*

For example, the following statement creates a function called add, which takes 2 parameters x and y:

add(x,y) := x + y

All input variables are not required to be used in the expression. However, it is strongly recommended to use all variables since they will be required when calling the function.

After being created, a user-defined function can be used like any other function. It is possible to define a function with the same name as a built-in function, but it cannot be called since the built-in function will take precedence.

User-Defined Variables

To create a user-defined variable, the following syntax is used:

[name] := *[expression]*

For example, the following statement creates a variable called phi:

phi := (sqrt(5)+1)/2

After being created, a user-defined variable can be used like any other variable. It is possible to define a variable with the same name as a built-in function, but it cannot be used since the built-in variable will take precedence.

Defining single-letter variables is not recommended, since it may interfere with the functionality of functions that target a variable in an expression.